

Amazon ML Challenge 2026 — Team A Playbook

Focus Area: S3 Central Data Store, Data Filtering, Normalization & Feature Pipeline

Operating Window: Friday 26th September (Full Day until 10:00 PM IST Hard Stop)

Target Milestone: Centralized S3 database populated with clean Parquet feature sets for Team B before exam day (27th Sept).

1. Executive Summary & Team Mission

Team A owns the data foundation for the entire hackathon.

Because AWS SageMaker compute quotas are constrained, **AWS is utilized exclusively for high-performance S3 cloud object storage (our central data lake / database)**. All heavy computation is run on **Google Colab (High-RAM/GPU) or local machines**, with S3 acting as the single source of truth across all teammates.

Your core mission on September 26:

- S3 Central Cloud Database Setup:** Standardize S3 bucket hierarchy for raw, filtered, candidate, and feature datasets.
 - Memory-Safe Data Filtering & Normalization:** Clean noisy business names, legal suffixes, addresses, and Hindi (Devanagari) script without OOM errors.
 - Smart Blocking:** Filter comparison space from 26.4 million potential pairs down to ~5\$ high-quality candidates per Source 1 entity (partitioned strictly by country).
 - Fast Similarity Feature Extraction:** Compute rapid string similarity scores (`rapidfuzz`) in memory-efficient batches.
 - Leak-Proof Train/Val Split:** Group by `source1_entity_id` to guarantee zero data leakage.
 - Central S3 Delivery by 10:00 PM:** Save final `.parquet` feature tables directly to S3 so Team B can train models on September 27.
-

2. Central S3 Storage Hierarchy (The Cloud Database)

All team members connect to the central S3 bucket: `s3://<your-bucket-name>/`

```

s3://<your-bucket-name>/
├─ raw/                                # Unprocessed source TSV files from Amazon
│   ├── train_source1.tsv
│   ├── train_source2.tsv
│   ├── train_source3.tsv
│   ├── train_ground_truth.tsv
│   ├── test_source1.tsv
│   ├── test_source2.tsv
│   └── test_source3.tsv
│
├─ filtered/                          # Cleaned, normalized, & filtered text data
│   ├── clean_train_s1.parquet
│   ├── clean_train_s23.parquet
│   ├── clean_test_s1.parquet
│   └── clean_test_s23.parquet
│
├─ candidates/                       # Blocked candidate pairs (country-partitioned)
│   ├── train_candidates_sample50k.parquet
│   ├── train_candidates_full.parquet
│   └── test_candidates_full.parquet
│
├─ features/                         # Extracted similarity features for modeling
│   ├── sample_50k/                  # Team B early-baseline test set
│   │   ├── train_candidates.parquet
│   │   └── val_candidates.parquet
│   └── full/                       # Full competition training set
│       ├── train_candidates.parquet
│       ├── val_candidates.parquet
│       └── test_candidates.parquet
│
├─ models/                          # Trained LightGBM model weights and configs
└─ submissions/                     # Generated TSVs & official submission zips

```

3. Team A Member Roles & Schedule

Member Role	Primary Focus	Key Responsibilities
Shubham (Lead)	S3 Architecture & Blocking	S3 pipeline orchestration, inverted index blocking, country partitioning, S3 exports
Member 2 (Data Partner)	Normalization & Features	String normalization, Devanagari text handling, Rapidfuzz feature extraction, data filtering

Timeline for Sept 26

- **09:00 AM - 10:00 AM:** Configure S3 bucket, test Colab/Local read/write permissions via `s3fs` and `boto3`.
- **10:00 AM - 01:00 PM: MILESTONE 1 (50k Sample):** Run filtering, blocking, and feature extraction on 50,000 entities. Upload to `s3://<bucket>/features/sample_50k/` for Team B.
- **02:00 PM - 06:00 PM:** Full-scale country-partitioned blocking on train & test sets; export candidate pairs to `s3://<bucket>/candidates/`.

- **06:00 PM - 09:30 PM:** Streaming feature extraction across all candidates in batches.
- **09:30 PM - 10:00 PM: MILESTONE 2 (Full Data):** Save full Parquets to `s3://<bucket>/features/full/`.
Handoff complete!

4. Step-by-Step Implementation Guide

Step 0: Environment & Central S3 Access Setup

(Run in Google Colab or your Local Terminal)

Install required packages:

```
pip install boto3 s3fs pyarrow duckdb rapidfuzz lightgbm scikit-learn tqdm
```

Authenticate with AWS S3:

```
import os
import boto3
import s3fs

# In Colab: Store these in Colab Secrets (Key icon on left) or set as environment variables
AWS_ACCESS_KEY = os.environ.get("AWS_ACCESS_KEY_ID", "YOUR_KEY_HERE")
AWS_SECRET_KEY = os.environ.get("AWS_SECRET_ACCESS_KEY", "YOUR_SECRET_HERE")
AWS_REGION = os.environ.get("AWS_DEFAULT_REGION", "us-east-1")
BUCKET_NAME = "your-hackathon-bucket"

# Initialize s3fs filesystem for direct pandas/pyarrow read/write
fs = s3fs.S3FileSystem(key=AWS_ACCESS_KEY, secret=AWS_SECRET_KEY)
print("S3 connection ready!")
```

Step 1: Text Normalization & Data Filtering

Indian records include Hindi Devanagari text (e.g., राम मार्केटिंग), and test data contains French names. Normalize Unicode, strip punctuation, standardize business suffixes, and filter uninformative noise:

```

import re
import unicodedata

def clean_text(text: str) -> str:
    if not isinstance(text, str) or not text.strip():
        return ""
    # Unicode NFKD normalization (handles Devanagari & French accents cleanly)
    text = unicodedata.normalize('NFKD', text).lower().strip()

    # Standardize business entity designations
    text = re.sub(r'\b(pvt|private)\b', 'pvt', text)
    text = re.sub(r'\b(ltd|limited)\b', 'ltd', text)
    text = re.sub(r'\b(inc|incorporated)\b', 'inc', text)
    text = re.sub(r'\b(corp|corporation)\b', 'corp', text)
    text = re.sub(r'\b(llc|llp)\b', 'llc', text)
    text = re.sub(r'\b&\b', 'and', text)

    # Address abbreviations
    text = re.sub(r'\b(rd|road)\b', 'road', text)
    text = re.sub(r'\b(st|street)\b', 'street', text)
    text = re.sub(r'\b(ave|avenue)\b', 'avenue', text)
    text = re.sub(r'\b(blvd|boulevard)\b', 'blvd', text)

    # Retain alphanumeric characters, whitespace, and Hindi Devanagari Unicode range
    text = re.sub(r'^\w\s\u0900-\u097F$', ' ', text)
    return re.sub(r'\s+', ' ', text).strip()

```

Save cleaned records directly to the S3 central lake:

```

# Save filtered clean tables to S3
clean_s1_df.to_parquet(f"s3://{BUCKET_NAME}/filtered/clean_train_s1.parquet", storage_options={"key": AWS_ACCESS_KEY_ID, "secret": AWS_SECRET_ACCESS_KEY})
clean_s23_df.to_parquet(f"s3://{BUCKET_NAME}/filtered/clean_train_s23.parquet", storage_options={"key": AWS_ACCESS_KEY_ID, "secret": AWS_SECRET_ACCESS_KEY})

```

Step 2: Country-Partitioned Inverted Index Blocking

Rule: Never compare across different countries (US vs IN vs FR).

Build an inverted index on significant tokens (≥ 3 characters) within each country to retrieve top ≤ 5 candidates:

```

from collections import defaultdict
import pandas as pd

def build_candidates_for_country(s1_df: pd.DataFrame, s23_df: pd.DataFrame, max_candidates: int = 5) -> pd.DataFrame:
    # 1. Build inverted index: token -> list of S2/S3 row indices
    index = defaultdict(list)
    for idx, name in enumerate(s23_df['clean_name']):
        for token in set(name.split()):
            if len(token) >= 3:
                index[token].append(idx)

    # 2. Match S1 records against inverted index
    candidate_pairs = []
    for _, s1 in s1_df.iterrows():
        s1_id = s1['entity_id']
        tokens = [t for t in set(s1['clean_name'].split()) if len(t) >= 3]

        counts = defaultdict(int)
        for t in tokens:
            for s23_idx in index.get(t, []):
                counts[s23_idx] += 1

        if not counts:
            continue

        top_matches = sorted(counts.items(), key=lambda x: x[1], reverse=True)[:max_candidates]
        for s23_idx, _ in top_matches:
            candidate_pairs.append({
                'source1_entity_id': s1_id,
                'candidate_entity_id': s23_df.iloc[s23_idx]['entity_id'],
                's1_name': s1['clean_name'],
                's23_name': s23_df.iloc[s23_idx]['clean_name'],
                's1_addr': s1['clean_addr'],
                's23_addr': s23_df.iloc[s23_idx]['clean_addr'],
                'country': s1['country']
            })
    return pd.DataFrame(candidate_pairs)

```

Save candidate pairs directly to S3:

```

candidate_df.to_parquet(
    f"s3://{BUCKET_NAME}/candidates/train_candidates_full.parquet",
    storage_options={"key": AWS_ACCESS_KEY, "secret": AWS_SECRET_KEY}
)

```

Step 3: Feature Extraction (`src/features.py`)

Run the vectorized C++ RapidFuzz engine to compute similarity scores, using float32 / int16 downcasting to keep memory low:

- `name_ratio`, `name_partial`, `name_token_sort`, `name_token_set`
- `addr_ratio`, `addr_partial`, `addr_token_set`
- `name_len_diff`, `addr_len_diff`

```
from src.features import compute_chunk_features, load_ground_truth_set

true_pairs = load_ground_truth_set("s3://<bucket>/raw/train_ground_truth.tsv") # or local copy
feat_chunk = compute_chunk_features(candidate_chunk, true_pairs=true_pairs)
```

Step 4: Leak-Proof Train/Val Split & S3 Delivery

Group strictly by `source1_entity_id` so the same business entity never leaks across training and validation:

```
from sklearn.model_selection import GroupShuffleSplit

gss = GroupShuffleSplit(n_splits=1, test_size=0.20, random_state=42)
train_idx, val_idx = next(gss.split(features_df, groups=features_df['source1_entity_id']))

train_data = features_df.iloc[train_idx].reset_index(drop=True)
val_data = features_df.iloc[val_idx].reset_index(drop=True)

# Export directly to central S3 feature store
storage_opts = {"key": AWS_ACCESS_KEY, "secret": AWS_SECRET_KEY}
train_data.to_parquet(f"s3://{BUCKET_NAME}/features/full/train_candidates.parquet", index=False, storage_options=storage_opts)
val_data.to_parquet(f"s3://{BUCKET_NAME}/features/full/val_candidates.parquet", index=False, storage_options=storage_opts)

print("✅ Team A mission complete! Feature store uploaded to central S3.")
```

5. Team A Golden Rules & Anti-Mistakes

- **DO NOT** use SageMaker notebook instances (quotas are 0). Use Google Colab or your local machine with S3 credentials.
- **DO NOT** save datasets into git repository. Always sync with `s3://<bucket>/`.
- **DO NOT** read TSVs without `sep="\t"`.
- **DO NOT** perform random row-based train/val splits. Group by `source1_entity_id`.
- **DO NOT** compare entities across different countries.
- **DO NOT** wait until night to give Team B data. Always deliver the 50,000 sample by 1:00 PM!